

vsock

一、概要

vsock 全称是 **Virtio Sockets**，是一种用于 **虚拟机和宿主机之间**（或者虚拟机和虚拟机之间）通信的机制。

和常见的 tcp/ip socket 不同，vsock 不依赖网络协议栈，而是通过虚拟化平台（如 KVM、QEMU、VMware、Hyper-V 等）提供的 **VirtIO 通道**直接通信。

特点

1. 点对点通信

它主要用于 **VM ↔ Host**、**VM ↔ VM**，而不是 VM ↔ 外部网络。

2. 简化通信

不需要 IP 地址、路由、NAT 等配置，只需要一个 **CID（Context ID，类似节点 ID）** 和 **端口号** 即可。

3. 性能较好

因为绕过了 TCP/IP 协议栈，延迟低，开销小，适合在虚拟化环境里做控制通道。

二、基本使用

vsock以一个PCI设备挂载给虚机使用，总体上经由guest层驱动 **virtio-vsock** 和host层驱动 **vhost-vsock** 两部分分别处理该设备的IO。二者均是宿主机和Guest上的内核模块。Host、Guest的Kernel版本都必须4.8以上，同时Qemu版本2.8以上。

两种 vsock 后端模式

QEMU 给 Guest 提供 virtio-vsock 设备时，可以选择 **谁来处理 virtqueue**：

• 用户态模式 (qemu-vsock)

- QEMU 自己在用户态处理 virtqueue，数据收发全走 QEMU 进程。
- 不需要宿主机的 vhost_vsock 内核模块。
- 缺点：性能较差，频繁用户态/内核态切换。

• 内核加速模式 (vhost-vsock)

- 使用宿主机内核的 vhost_vsock 驱动来处理数据路径。
- QEMU 只负责控制面（创建设备、管理配置）。
- 性能更好，延迟更低。

XML示例：

```
1 <devices>
2   <vsock model="virtio">
3     <!-- 自动分配 Guest CID -->
4     <cid auto="yes"/>
5     <!-- 可选：绑定固定 CID（需全局唯一） -->
6     <!-- <cid address="3"/> -->
7   </vsock>
8 </devices>
9
10 qemu-system-x86_64 -m 4G -hda /home/matt/ubuntuvm0.img \
11     -device vhost-vsock-pci,id=vhost-vsock-pci0,guest-cid=3 -vnc :0 \
12     --enable-kvm
```

Host端监听，类似于一个Web服务端：

```
1 #include <sys/socket.h>
2 #include <linux/vm_sockets.h>
3 #include <string.h>
4 #include <stdio.h>
5
6 int main()
7 {
8     int s = socket(AF_VSOCK, SOCK_STREAM, 0);
9
10    struct sockaddr_vm addr;
11    memset(&addr, 0, sizeof(struct sockaddr_vm));
12    addr.svm_family = AF_VSOCK;
13    addr.svm_port = 9999;
14    addr.svm_cid = VMADDR_CID_HOST;
15
16    bind(s, &addr, sizeof(struct sockaddr_vm));
17
18    listen(s, 0);
19
20    struct sockaddr_vm peer_addr;
21    socklen_t peer_addr_size = sizeof(struct sockaddr_vm);
22    int peer_fd = accept(s, &peer_addr, &peer_addr_size);
23
24    char buf[64];
25    size_t msg_len;
26    while ((msg_len = recv(peer_fd, &buf, 64, 0)) > 0) {
27        printf("Received %lu bytes: %.*s\n", msg_len, msg_len, buf);
28    }
29
30    return 0;
31 }
```

Guest端请求，类似于Web客户端：

```
1 #include <sys/socket.h>
2 #include <linux/vm_sockets.h>
3 #include <string.h>
4
5 int main()
6 {
7     int s = socket(AF_VSOCK, SOCK_STREAM, 0);
8
9     struct sockaddr_vm addr;
10    memset(&addr, 0, sizeof(struct sockaddr_vm));
11    addr.svm_family = AF_VSOCK;
12    addr.svm_port = 9999;
13    addr.svm_cid = VMADDR_CID_HOST;
14
15    connect(s, &addr, sizeof(struct sockaddr_vm));
16
17    send(s, "Hello, world!", 13, 0);
18
19    close(s);
20
21    return 0;
22 }
23
```

三、使用场景

- 同一节点上的虚机不需要网络连接的通信；不适用跨节点的虚机通信，受物理网络限制
- 节点上的虚机不在同一子网，需要桥接才能访问
- 容器与Host通信，容器运行时（Containerd / Kata Containers）
- 实时日志采集，高吞吐、低延迟，避免日志堆积

四、内核socket

socket 本质上是内核暴露出来的一种通用接口机制。

1. socket 的最初设计

- **起源**：socket 最早在 BSD Unix 里提出，用来统一不同通信方式（网络、进程间通信）的编程接口。
- **本质**：socket 是 **内核里的一个抽象对象**，对应一组操作方法（创建、读写、关闭...），用户态通过系统调用 socket()、bind()、connect() 等去使用。
- **作用**：给用户态一个统一的“文件描述符”接口，把各种内核通信机制包装成像读写文件一样的操作。

2. Address Family (AF)

- 在实际应用中，最常见的 socket 类型是 **AF_INET / AF_INET6**（TCP/IP 网络通信）。
- TCP/IP 成为互联网的核心协议，导致“socket = 网络套接字”这种印象逐渐固定。
- 但实际上，socket 族（Address Family）远不止网络：
 - AF_UNIX：Unix 域 socket，本地进程间通信，例如MySQL、Nginx本地进程通信
 - AF_VSOCK：虚拟机 ↔ 宿主机/虚拟机通信
 - AF_ALG：Linux 内核的加密算法接口
 - AF_NETLINK：用户态 ↔ 内核通信（路由表、网络配置等）
 - AF_XDP：用户态直接访问网卡收发队列

3. 内核接口视角

所以更准确的说法是：

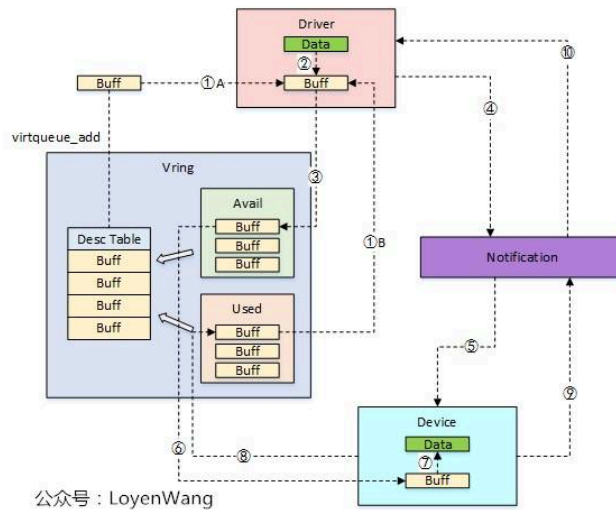
- **socket 是 Linux 内核提供的一种 IPC（进程间通信）接口**，支持多种“地址族”。
- 网络只是其中最常见、最成功的应用场景。
- 用 socket(AF_INET, SOCK_STREAM, 0) → 就是网络
- 用 socket(AF_ALG, SOCK_SEQPACKET, 0) → 就是加密服务
- 用 socket(AF_VSOCK, SOCK_STREAM, 0) → 就是虚拟机通道

五、传输媒介 virtqueue

既然是一个virtio pci设备，那就要遵循virtqueue的规范，无论是G-->H还是H-->G，主旨都是通过共享内存交换数据，virtqueue主要是以下几部分：

- Guest data buffer
- descriptor
- vring（avail ring、used ring），取模
- 虚拟中断，eventfd、irqfd

从G-->H看一下virtqueue流程：



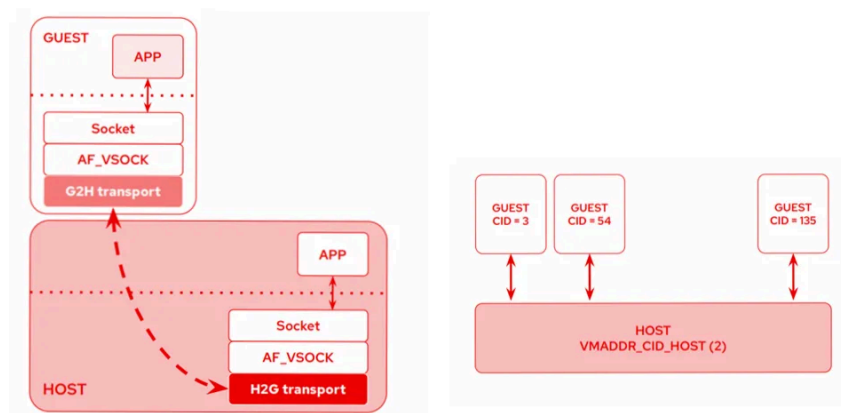
- ①A 表示发送方向，分配一个 buffer 并添加到 virtqueue 中，①B 表示接收方向，从 used ring 中获取一个buffer（表示IO完成，回收buffer），这两种中选择一种方式；
- ② 表示将数据拷贝到 buffer中，用于传送；
- ③ 表示更新 avail ring 中的描述符索引值，注意，驱动中需要执行 memory barrier 操作，大致作用是保证前后的内存操作顺序，确保 device 能看到正确的值；
- ④ 与 ⑤ 表示 driver 通知 device 来取数据（eventfd）；
- ⑥ 表示 device 从 avail ring 中获取到描述符索引值；
- ⑦ 表示将描述符索引对应的地址中的数据取出来；
- ⑧ 表示 device 更新 used 队列中的描述符索引；
- ⑨ 与 ⑩ 表示 device 通知 driver 数据已经取完了（发送虚拟中断 irqfd，回收data buffer）。

H-->G 和 G→H 差不多，只是方向相反：

Guest 提供空 buffer → 放 avail → Host 轮询/中断发现avail有增加，写数据进去 → 更新 used → 中断通知 → Guest 取数据

和蜂巢快递柜很像。

六、架构原理



G-->H :

1. qemu 启动加上 vhost-vsock-pci 参数，并指定虚拟机唯一的cid编号，从而host以及guest都有了各自的cid
2. Guest侧的应用程序创建一个VSOCK socket并连接server端(CID=2,port=1234)，其中CID=2 仅代表宿主机，将数据写入 socket缓冲区，virtio-vsock 驱动通过 Virtio 机制，将请求发送至virtqueues(ring buffer)。
3. 如果未启用vhost-vsock，qemu会直接读取 vring，解析 vsock packet，转发到 Host socket。如果启用vhost-vsock加速，QEMU 把 vring 的共享内存 fd 传给 vhost，将数据从共享内存复制到Host内核空间，并递交给Host应用。
4. host应用监听 AF_VSOCK socket，内核 vsock 模块收到数据后放入 socket buffer，host应用调用recv获取。

H-->G :

1. **Guest 驱动先准备 buffer**
 - Guest (virtio-vsock 驱动) 提前分配一批内存页作为 **接收缓冲区**。
 - 在 **descriptor table** 里登记这些 buffer 的地址和大小。
 - 把这些 descriptor 的索引写入 **avail ring**，表示有空 buffer，设备可以写数据进来。
2. **Host (vhost-vsock) 要发数据给 Guest**
 - 用户态应用（比如宿主机上的 socat 或管理工具）往 vsock socket 写数据。
 - vhost-vsock 内核模块作为“设备端”，从 **avail ring** 中取出一个 descriptor 索引。
 - 查到 descriptor → 找到 Guest 提供的 buffer（共享内存）。
3. **Host 把数据写到 Guest buffer**
 - vhost-vsock 把应用层写入的数据拷贝到 descriptor 指向的 buffer 里。
 - 写完后，设备更新 **used ring**，把这个 buffer 的索引写进去，表示“数据已经放好了”。
4. **Host 通知 Guest**
 - vhost-vsock 触发一次 **virtio 中断**（虚拟 IRQ），通知 Guest 驱动。
5. **Guest 驱动处理数据**
 - virtio-vsock 驱动收到中断 → 从 **used ring** 里取出已完成的 descriptor 索引。
 - 根据索引找到 buffer，数据已经在里面了。
 - 把数据递交给 Guest 内核的 vsock socket 层 → 最终交给 Guest 用户态应用。
6. **Buffer 回收再利用**
 - Guest 应用处理完数据，驱动把这个 buffer 重新挂回 **avail ring**，等待下一次写入。

七、Host是如何路由的

7.1 建立CID与virtqueue的映射关系

每个虚机的每个virtio设备都会至少创建一个virtqueue，每个 virtqueue 属于一个设备，不能跨设备共享。

• virtqueue 的数量和设备类型有关：

- **单向/双向数据流**：有些设备一个 virtqueue 就够了，有些设备需要 **一对 virtqueue**（发送/接收）
- **多队列 (multi-queue)**：比如 virtio-net 可以有多个 virtqueue，每个队列对应一条 CPU 或线程，提高并发性能

每个虚机的每个 virtio 设备都会建至少一个 virtqueue，有的双向或多队列设备会建多个 virtqueue。

- Host vhost-vsock 维护 CID ↔ virtqueue（或 guest session）的映射
- Qemu 维护了一张表，记录每个 virtqueue 的归属信息
- 有了这样的关系，双方对数据的存取地点就很清楚

virtqueue 地址/ID	Guest CID、port	设备类型	virtio 设备 ID	对应 vhost 驱动
0xabc123	3:9999	vsock	vhost-vsock0	vhost-vsock
0xdef456	-	blk	vhost-blk0	vhost-blk

7.2 Guest生成vsock报文

- 1.这个驱动会把socket调用转换成virtqueue操作
- 2.给数据加上vsock报文头，再加上数据本身payload

7.3 Guest驱动利用virtqueue的机制传输

7.4 Host读取报文并搬运

Host 是必要的中介，因为 virtqueue 是每个 virtio 设备私有的，没有跨 Guest 共享。Host根据源virtqueue里的目标cid映射到目标virtqueue，Host 查映射表，找到：

1. 这个 virtqueue 属于哪个 Guest
2. 是什么类型的 virtio 设备
3. 对应的 vhost 驱动

搬运到目标virtqueue相当于H-->G的流程，最终更新目标virtqueue的used ring，发送中断通知目标内核，内核调用virtio-vsock驱动。

7.5 目标驱动读取并解析报文

从 **used ring** 获取 descriptor 索引，根据 descriptor 指向的内存地址读取 vsock 报文。

7.6 将数据放入socket队列

驱动根据 **socket 映射表**（端口 + CID）找到对应的 AF_VSOCK socket，将数据payload放入socket内核缓冲区队列等待 receive。

总体职责：

virtio-vsock客户端：

- 1.负责生成vsock报文
- 2.负责将报文丢入virtqueue的descriptor指向的data buffer
- 3.更新 avail ring，通知 Host/vhost-vsock 有新数据
- 4.触发 kick/notify eventfd，Host 可以取走数据

virtio-vsock服务端：

- 1.监听port
- 2.维护pending queue

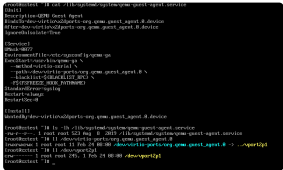
vhost-sock：

- 1.读取virtqueue，获取vsock报文

2.路由，根据报文的目标 CID，将数据写入目标 Guest 的 virtqueue 或 Host 用户态 socket

八、与virtio-serial区别

virtio-serial的Unix Socket设备如下图所示，Host通过该设备与QGA交互。



相同点：

- 都是 **virtio 设备**，挂在 Guest PCI 总线上。
- 都提供 **Guest ↔ Host 通信通道**，不依赖物理网卡。
- 数据传输都走 **virtqueue（环形缓冲区）**。
- 都可以通过一定的方式提供用户态访问接口，vsock是通过socket方式，virtio-serial是通过字符设备文件的方式。

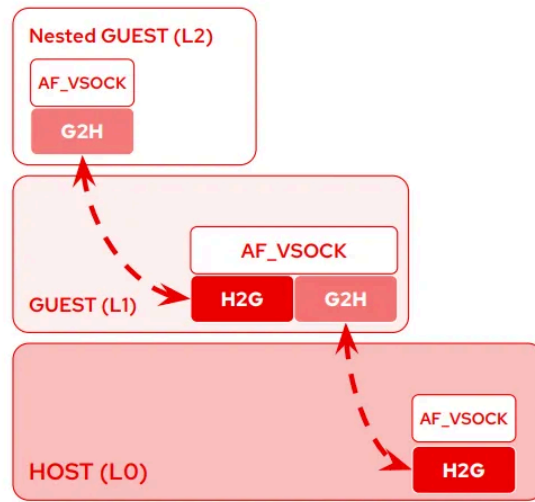
不同点：

项目	virtio-serial	vsock
接口类型	字符设备 (/dev/virtio-ports/...)	Socket API (AF_VSOCK)
通信语义	简单字节流，需要应用自己处理 framing	Stream / Datagram，内核提供完整协议栈
使用范围	通常 Guest ↔ Host（单点）	Guest ↔ Host，Guest ↔ Guest（同宿主机）
应用场景	控制类、管理类小数据量通道（如 QEMU Guest Agent）	大规模数据传输、应用间通信
性能优化	简单，少量数据即可	支持 vhost-vsock 内核加速，适合高性能通信
数据处理	应用层需自己解析消息边界	内核协议栈Datagram处理消息边界，类似 TCP/UDP，贴近聊天记录

virtio-serial必须由host手动做用户态转发，virtio-sock 内置 CID 路由定位。

九、Kata

同时目前vsock已经对多层次通信进行了支持，支持guest中和nested guest的通信，在高版本内核5.5+支持L1 GUEST同时加载G2H以及H2G transport，可以和L0与L2通信，从而支持了kata等容器场景。



十、友商？

VMWare VMTools

参考链接

<https://mp.weixin.qq.com/s/HYRFdGNMpYMZvyQTDXHHQA>

 [I/O 虚拟化（一）：Split Virtqueue](#)